

ANÁLISIS DEL RETO

Samuel Casallas, 202220537, s.casallas1@uniandes.edu.co

Federico Mateo Castro Fernandez, 202510264, fm.castrof1@uniandes.edu.co

Samuel Barragán, 202420209, s.barragang@uniandes.edu.co

Requerimiento <<1>>

Descripción

Este requerimiento recorre una sola vez el array_list que contiene todos los pedidos cargados, comparando el campo "Product" de cada uno con el producto ingresado. Mientras avanza, va acumulando las sumas de las variables Price_per_Box, Discount_Pct, Boxes_Shipped y Marketing_Spend, para poder calcular promedios, el mínimo y máximo de cada característica; también cuenta los pedidos en cada año y determina el año con más pedidos.

Entrada	Catálogo de pedidos, nombre del producto a buscar
Salidas	Cantidad de pedidos del producto, promedio/mínimo/máximo de Price_per_Box, Discount_Pct, Boxes_Shipped y Marketing_Spend, año con más pedidos, mayor y menor pedido (Amount). Si no existen pedidos con el producto, retorna None
Implementado (Sí/No)	Sí, realizado por Samuel Casallas

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer array_list por get_element()	$O(n)$
Por cada pedido: comparar producto, acumular sumas y actualizar info. de máximos y mínimos	$O(n)$
Calcular los 4 promedios	$O(1)$
Determinar el año con más pedidos	$O(m)$ donde m =años distintos
TOTAL	$O(n)$

Análisis

Como no hay forma previa de saber cuáles pedidos corresponden al producto buscado, es necesario revisar los n pedidos del catálogo al menos una vez. El acceso por sí solo al catálogo tiene una complejidad temporal constante; el recorrido completo será $O(n)$. Las operaciones dentro del recorrido tampoco afectan ni varían la complejidad, ya que son constantes.

Requerimiento <<2>>

Descripción

Recorre una sola vez el `array_list` con `get_element`, comparando `Price_per_Box` contra el rango `[min_price, max_price]`. Por cada pedido dentro del rango, acumula las sumas necesarias para los tres promedios y actualiza tres pedidos extremos (más reciente por `Order_Date`, menor `Amount`, mayor `Amount`), aplicando los criterios de desempate definidos en el enunciado.

Entrada	Catálogo de pedidos, precio mínimo por caja, precio máximo por caja
Salidas	Cantidad de pedidos en el rango; promedio de <code>Discount_Pct</code> , <code>Marketing_Spend</code> y <code>Price_per_Box</code> ; pedido más reciente; pedido de menor <code>Amount</code> ; pedido de mayor <code>Amount</code> . Si no hay pedidos en el rango, <code>count=0</code>
Implementado (Sí/No)	Sí, implementado por Federico Mateo Castro Fernandez

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer <code>array_list</code> con <code>get_element()</code>	$O(n)$
Por cada pedido en rango: comparar precio, acumular sumas y actualizar los 3 extremos	$O(n)$
Calcular los 3 promedios finales	$O(1)$
TOTAL	$O(n)$

Análisis

Como no hay ninguna estructura auxiliar que indique de antemano cuáles pedidos caen en el rango de precio, es necesario revisar los n pedidos del catálogo al menos una vez. El acceso a cada posición del `array_list` vía `get_element` es $O(1)$, así que el recorrido completo es $O(n)$. Las comparaciones dentro del ciclo (mínimo, máximo, más reciente) son de costo constante y no cambian el orden del algoritmo.

Requerimiento <<3>>

Descripción

Recorre una sola vez el `array_list`, comparando `Country` y `Channel` de cada pedido contra el país y canal ingresados. Por cada pedido que cumple el filtro, acumula las sumas de `Price_per_Box`, `Discount_Pct`,

Marketing_Spend y Boxes_Shipped, y usa dos diccionarios auxiliares (contador_producto y contador_ano) para contar cuántas veces aparece cada producto y cada año entre los pedidos filtrados.

Entrada	Catálogo de pedidos, país, canal
Salidas	Cantidad de pedidos que cumplen el filtro; promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped; producto más frecuente; año con más pedidos. Si no hay pedidos con esa combinación, retorna None
Implementado (Sí/No)	Sí, implementado por Samuel Barragán

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer array_list con get_element()	$O(n)$
Por cada pedido: comparar país/canal, acumular sumas y actualizar los contadores	$O(n)$
Calcular los 4 promedios	$O(1)$
Determinar producto más frecuente y año con más pedidos (max() sobre los diccionarios)	$O(k)$, k = productos/años distintos
TOTAL	$O(n)$

Análisis

El recorrido principal es $O(n)$ porque hay que revisar cada pedido para verificar si coincide con el país y canal buscados. Las operaciones sobre los diccionarios son $O(1)$ amortizado. Al final, max() sobre los contadores solo recorre las llaves distintas (k productos o años distintos), y como k nunca es mayor que n , esta parte queda dominada por el recorrido principal; la complejidad total es $O(n)$.

Requerimiento <<4>>

Descripción

Este requerimiento se resuelve en dos recorridos: el primero pasa por el array_list donde se tiene cada producto y filtra los que cumplen las condiciones (producto y país), agregándolos al final de una lista enlazada. En el segundo recorrido se analiza la lista filtrada nodo por nodo, utilizando las características de "first" y "next"; mediante este avance se guarda y acumula las sumas de las variables necesarias, y finalmente se determina, con una función auxiliar (es_mejor), los pedidos de mayor Amount.

Entrada	Catálogo de pedidos, nombre de producto, país
Salidas	Cantidad de pedidos, promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, los dos pedidos de mayor Amount (top1 y top2). Si no hay pedidos con esa combinación, retorna None

Implementado (Sí/No)	Sí, implementado por Samuel Casallas
-----------------------------	--------------------------------------

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorre el array_list y filtra por producto y país a buscar	$O(n)$
Recorrer la single_linked con m pedidos filtrados avanzando nodo por nodo y guardando la información necesaria	$O(m)$
Calcular los 4 promedios finales	$O(1)$
TOTAL	$O(n+m)$

Análisis

El primer recorrido es lineal porque el acceso por índice en el array_list es $O(1)$; sin embargo, al hacer un recorrido completo de todos los productos y países encontrando las combinaciones, se vuelve una complejidad lineal. Por otra parte, el segundo recorrido es sobre la lista enlazada, avanzando con el nodo next, por lo que también se vuelve una complejidad lineal. Se podría analizar que, como m no puede ser mayor a n (ya que después de recorrer n se obtuvo un filtro considerado como m), la complejidad total dependiendo del tamaño estaría como $n > m$, donde prevalece una complejidad $O(n)$.

Requerimiento <<5>>

Descripción

Recorre la lista enlazada nodo por nodo (con "first" y "next", sin reiniciar desde el principio en cada paso) filtrando los pedidos cuyo producto coincide y cuya fecha cae en el rango dado. Acumula las sumas para los tres promedios y compara el pedido actual contra el mejor encontrado hasta el momento, según el filtro MENOR/MAYOR y los criterios de desempate.

Entrada	Catálogo de pedidos, filtro (MENOR o MAYOR), producto, fecha inicial, fecha final
Salidas	Filtro usado, cantidad de pedidos que cumplen, promedio de Price_per_Box/Boxes_Shipped/Marketing_Spend, y el pedido con el Amount menor o mayor según el filtro. Si no hay pedidos, count=0
Implementado (Sí/No)	Sí, implementado por Federico Mateo Castro Fernandez

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer la single_linked_list nodo por nodo con next	$O(n)$

Por cada nodo: comparar producto/fecha, acumular sumas y comparar contra target_order	$O(n)$
Calcular los 3 promedios finales	$O(1)$
TOTAL	$O(n)$

Análisis

La primera versión de este requerimiento recorría la lista llamando `get_element(lista, i)` dentro de un `for`; como `get_element` camina desde el primer nodo en cada llamada, eso costaba $O(n^2)$ en vez de $O(n)$. Se corrigió recorriendo la lista una sola vez con un puntero nodo que avanza con `nodo["next"]`, igual que hace req_4. Con esa corrección, el recorrido es una sola pasada de $O(n)$, y las comparaciones dentro de cada iteración son de costo constante, así que la complejidad total queda en $O(n)$.

Requerimiento <<6>>

Descripción

Recorre la lista enlazada una sola vez, nodo por nodo, filtrando los pedidos cuya fecha cae en el rango dado. Por cada pedido que cumple, actualiza en un diccionario canales (agrupado por Channel) el número de pedidos, el recaudo acumulado, las sumas para promedios, y el pedido más barato/costoso de ese canal. Al final determina, con `max()` sobre ese diccionario, el canal con más pedidos y el canal con mayor recaudo, y arma el detalle por canal.

Entrada	Catálogo de pedidos, fecha inicial, fecha final
Salidas	Cantidad total de pedidos en el rango; canal más usado (con su total de pedidos y recaudo); canal que más recauda (con su total de pedidos y recaudo); y por cada canal presente en el rango: precio promedio, inversión en mercadeo promedio, y su pedido más costoso y más barato
Implementado (Sí/No)	Sí, implementado por Samuel Barragán

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrer la <code>single_linked_list</code> nodo por nodo con <code>next</code>	$O(n)$
Por cada nodo dentro del rango: actualizar el diccionario canales y comparar contra <code>min_order</code> / <code>max_order</code>	$O(n)$
Determinar <code>canal_mas_usado</code> y <code>canal_mas_recauda</code> con <code>max()</code> sobre canales	$O(c)$, c = canales distintos (máx. 3)
Armar <code>detalle_canales</code> recorriendo <code>canales.items()</code>	$O(c)$
TOTAL	$O(n)$

Análisis

Al igual que en req_5, la primera versión usaba `get_element(lista, i)` dentro de un `for`, lo que costaba $O(n^2)$. Se corrigió con el mismo patrón de recorrido por nodos (`first/next`) que ya usa req_4. El diccionario canales tiene como máximo 3 llaves porque Channel solo puede ser Retail, Online o Wholesale, así que las operaciones de `max()` y el armado del detalle final son $O(c)$ con $c \leq 3$ —en la práctica una constante— y no cambian el orden general del algoritmo, que queda en $O(n)$.